Объектно-ориентированное программирование

6 семестр
Лекция 3.

Содержание лекции:

- Основы паттернов
- Группы паттернов
- Паттерн Стратегия
- Паттерн MVC

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Основы паттернов

Паттерн представляет определенный способ построения программного кода для решения часто встречающихся проблем проектирования. В данном случае предполагается, что есть некоторый набор общих формализованных проблем, которые довольно часто встречаются, и паттерны предоставляют ряд принципов для решения этих проблем.

Что же дает нам применение паттернов? При написании программ мы можем формализовать проблему в виде классов и объектов и связей между ними. И применить один из существующих паттернов для ее решения. В итоге нам не надо ничего придумывать. У нас уже есть готовый шаблон, и нам только надо его применить в конкретной программе.

Паттерны, как правило, **не зависят от языка программирования**. Их принципы применения будут аналогичны и в C#, и в Jave, и в других языках. Хотя в рамках данного руководства мы будем говорить о паттернах в контексте языка C#.

Также мышление паттернами упрощает групповую разработку программ. Зная применяемый паттерн проектирования и его основные принципы другому программисту будет проще понять его реализацию и использовать ее.

В то же время **не стоит применять паттерны ради самих паттернов**. Хорошая программа предполагает использование паттернов. Однако не всегда паттерны упрощают и улучшают программу. Неоправданное их использование может привести к усложнению программного кода, уменьшению его качества. Паттерн должен быть оправданным и эффективным способом решения проблемы.

Существует множество различных паттернов, которые решают разные проблемы и выполняют различные задачи. Но по своему действию их можно объединить в ряд групп. Рассмотрим некоторые группы паттернов. В основу классификации основных паттернов положена цель или задачи, которые определенный паттерн выполняет.

Группы паттернов

Порождающие паттерны — это паттерны, которые абстрагируют процесс инстанцирования или, иными словами, процесс порождения классов и объектов. Среди них выделяются следующие:

- **Абстрактная фабрика (Abstract Factory)**
- **Строитель (Builder)**
- **Фабричный метод (Factory Method)**
- **Прототип (Prototype)**
- **Одиночка (Singleton)**

Структурные паттерны - рассматривает, как классы и объекты образуют более крупные структуры - более сложные по характеру классы и объекты. К таким шаблонам относятся:

- **Адаптер (Adapter)**
- **Мост (Bridge)**
- **Компоновщик (Composite)**
- **Декоратор (Decorator)**
- **Фасад (Facade)**
- **Приспособленец (Flyweight)**
- **Заместитель (Proxy)**

Поведенческие паттерны - они определяют алгоритмы и взаимодействие между классами и объектами, то есть их поведение. Среди подобных шаблонов можно выделить следующие:

- **Цепочка обязанностей (Chain of responsibility)**
- **Команда (Command)**
- **Интерпретатор (Interpreter)**
- **Итератор (Iterator)**
- **Посредник (Mediator)**
- **Хранитель (Memento)**
- **Наблюдатель (Observer)**
- **Состояние (State)**
- **Стратегия (Strategy)**
- **Шаблонный метод (Template method)**
- **Посетитель (Visitor)**

Как выбрать нужный паттерн?

- Прежде всего при решении какой-нибудь проблемы надо выделить все используемые сущности и связи между ними и абстрагировать их от конкретной ситуации.
- Затем надо посмотреть, вписывается ли абстрактная форма решения задачи в определенный паттерн. Например, суть решаемой задачи может состоять в создании новых объектов. В этом случае, возможно, стоит посмотреть на порождающие паттерны.
- Причем лучше не сразу взять какой-то определенный паттерн - первый, который показался нужным, а посмотреть на несколько родственных паттернов из одной группы, которые решают одну и ту же задачу.

Важно понимать смысл и назначение паттерна, явно представлять его абстрактную организацию и его возможные конкретные реализации. Один паттерн может иметь различные реализации, и чем чаще вы будете сталкиваться с этими реализациями, тем лучше вы будете понимать смысл паттерна. Но не стоит использовать паттерн, если вы его не понимаете, даже если он на первый взгляд поможет вам в решении задачи.

И в конечном счете надо придерживаться принципа **KISS (Keep It Simple, Stupid)** - сохранять код программы по возможности простым и ясным. Ведь смысл паттернов не в усложнении кода программы, а наоборот в его упрощении.

Порождающие паттерны

Фабричный метод (Factory Method)

это паттерн, который определяет интерфейс для создания объектов некоторого класса, но непосредственное решение о том, объект какого класса создавать происходит в подклассах. То есть паттерн предполагает, что базовый класс делегирует создание объектов классам-наследникам.

Когда надо применять паттерн

- Когда заранее неизвестно, объекты каких типов необходимо создавать
- Когда система должна быть независимой от процесса создания новых объектов и расширяемой: в нее можно легко вводить новые классы, объекты которых система должна создавать.
- Когда создание новых объектов необходимо делегировать из базового класса классам наследникам

Допустим, мы создаем программу для сферы строительства. Возможно, вначале мы захотим построить многоэтажный панельный дом. И для этого выбирается соответствующий подрядчик, который возводит каменные дома. Затем нам захочется построить деревянный дом и для этого также надо будет выбрать нужного подрядчика.

```
abstract class House { }

class PanelHouse : House{
    public PanelHouse() =>
        Console.WriteLine("Панельный дом построен");
}
class WoodHouse : House{
    public WoodHouse() =>
        Console.WriteLine("Деревянный дом построен");
}
```

```
// абстрактный класс строительной компании
abstract class Developer{
    public string Name { get; set; }
    public Developer(string n) => Name = n;
    // фабричный метод
    abstract public House Create();
}
class PanelDeveloper : Developer{ // строит панельные дома
    public PanelDeveloper(string n) : base(n) { }

    public override House Create() => new PanelHouse();

}
class WoodDeveloper : Developer{ // строит деревянные дома
    public WoodDeveloper(string n) : base(n) { }

    public override House Create() => new WoodHouse();
}
```

```
Developer dev = new PanelDeveloper("ООО КирпичСтрой");
House house2 = dev.Create();

dev = new WoodDeveloper("Частный застройщик");
House house = dev.Create();
Console.ReadLine();
```

В качестве абстрактного класса Product здесь выступает класс House. Его две конкретные реализации - PanelHouse и WoodHouse представляют типы домов, которые будут строить подрядчики. В качестве абстрактного класса создателя выступает Developer, определяющий абстрактный метод Create(). Этот метод реализуется в классах-наследниках WoodDeveloper и PanelDeveloper. И если в будущем нам потребуется построить дома какого-то другого типа, например, кирпичные, то мы можем с легкостью создать новый класс кирпичных домов, унаследованный от House, и определить класс соответствующего подрядчика. Таким образом, система получится легко расширяемой. Правда, недостатки паттерна тоже очевидны - для каждого нового продукта необходимо создавать свой класс создателя.

Одиночка (Singleton, Синглтон) - порождающий паттерн, который гарантирует, что для определенного класса будет создан только один объект, а также предоставит к этому объекту точку доступа.

Когда надо использовать Синглтон? Когда необходимо, чтобы для класса существовал только один экземпляр

Синглтон позволяет создать объект только при его необходимости. Если объект не нужен, то он не будет создан. В этом отличие синглтона от глобальных переменных.

```
class Computer
{
    public OS OS { get; set; }
    public void Launch(string osName)=> OS = OS.getInstance(osName);
}
class OS
{
    private static OS instance;
    public string Name { get; private set; }

    protected OS(string name) => this.Name = name;

    public static OS getInstance(string name){
        if (instance == null)
            instance = new OS(name);
        return instance;
    }
}
```

```
Computer comp = new Computer();
comp.Launch("Windows 8.1");
Console.WriteLine(comp.OS.Name);

// у нас не получится изменить ОС, так как объект уже создан
comp.OS = OS.getInstance("Windows 10");
Console.WriteLine(comp.OS.Name);

Console.ReadLine();
```

Паттерны поведения

Паттерн Стратегия (Strategy) представляет шаблон проектирования, который определяет набор алгоритмов, инкапсулирует каждый из них и обеспечивает их взаимозаменяемость. В зависимости от ситуации мы можем легко заменить один используемый алгоритм другим. При этом замена алгоритма происходит независимо от объекта, который использует данный алгоритм.

Когда использовать стратегию?

- Когда есть несколько родственных классов, которые отличаются поведением. Можно задать один основной класс, а разные варианты поведения вынести в отдельные классы и при необходимости их применять
- Когда необходимо обеспечить выбор из нескольких вариантов алгоритмов, которые можно легко менять в зависимости от условий
- Когда необходимо менять поведение объектов на стадии выполнения программы
- Когда класс, применяющий определенную функциональность, ничего не должен знать о ее реализации

Пример. Существуют различные легковые машины, которые используют разные источники энергии: электричество, бензин, газ и так далее. Есть гибридные автомобили. В целом они похожи и отличаются преимущественно видом источника энергии. Не говоря уже о том, что мы можем изменить применяемый источник энергии, модифицировав автомобиль.

```
interface Imovable {
    void Move();
}
class PetrolMove : Imovable
{
    public void Move() => Console.WriteLine("Перемещение на бензине");
}
class ElectricMove : Imovable
{
    public void Move()
     => Console.WriteLine("Перемещение на электричестве");
}
```

```
class Car
{
    protected int passengers; // кол-во пассажиров
    protected string model; // модель автомобиля

    public Car(int num, string model, IMovable mov){
        this.passengers = num;
        this.model = model;
        Movable = mov;
    }
    public IMovable Movable { private get; set; }
    public void Move() => Movable.Move();
}
```

```
Car auto = new Car(4, "Volvo", new PetrolMove());
auto.Move();
auto.Movable = new ElectricMove();
auto.Move();

Console.ReadLine();
```

В данном случае в качестве IStrategy выступает интерфейс IMovable, определяющий метод Move(). А реализующий этот интерфейс семейство алгоритмов представлено классами ElectricMove и PetroleMove. И данные алгоритмы использует класс Car.

Структурные паттерны

Декоратор (Decorator) представляет структурный шаблон проектирования, который позволяет динамически подключать к объекту дополнительную функциональность.

Для определения нового функционала в классах нередко используется наследование. Декораторы же предоставляет наследованию более гибкую альтернативу, поскольку позволяют динамически в процессе выполнения определять новые возможности у объектов.

Когда следует использовать декораторы?

- Когда надо динамически добавлять к объекту новые функциональные возможности. При этом данные возможности могут быть сняты с объекта
- Когда применение наследования неприемлемо. Например, если нам надо определить множество различных функциональностей и для каждой функциональности наследовать отдельный класс, то структура классов может очень сильно разрастись. Еще больше она может разрастись, если нам необходимо создать классы, реализующие все возможные сочетания добавляемых функциональностей.

Рассмотрим пример. Допустим, у нас есть пиццерия, которая готовит различные типы пицц с различными добавками. Есть итальянская, болгарская пиццы. К ним могут добавляться помидоры, сыр и т.д. И в зависимости от типа пицц и комбинаций добавок пицца может иметь разную стоимость. Теперь посмотрим, как это изобразить в программе на C#:

```
abstract class Pizza
{
    public Pizza(string n) => this.Name = n;

    public string Name { get; protected set; }
    public abstract int GetCost();
}

class ItalianPizza : Pizza
{
    public ItalianPizza() : base("Итальянская пицца") { }
    public override int GetCost() => return 10;
}
class BulgerianPizza : Pizza
{
    public BulgerianPizza() : base("Болгарская пицца") { }
    public override int GetCost() => return 8;
}
```

```
abstract class PizzaDecorator : Pizza
{
    protected Pizza pizza;
    public PizzaDecorator(string n, Pizza pizza) : base(n)
         => this.pizza = pizza;
}

class TomatoPizza : PizzaDecorator
{
    public TomatoPizza(Pizza p) : base(p.Name + ", с томатами", p) { }
    public override int GetCost() => return pizza.GetCost() + 3;
}
class CheesePizza : PizzaDecorator
{
    public CheesePizza(Pizza p) : base(p.Name + ", с сыром", p) { }
    public override int GetCost() => return pizza.GetCost() + 5;
}
```

```
Pizza pizza1 = new ItalianPizza();
pizza1 = new TomatoPizza(pizza1); // итальянская пицца с томатами
Console.WriteLine("Название: {0}", pizza1.Name);
Console.WriteLine("Цена: {0}", pizza1.GetCost());

Pizza pizza2 = new ItalianPizza();
pizza2 = new CheesePizza(pizza2);// итальянская пиццы с сыром
Console.WriteLine("Название: {0}", pizza2.Name);
Console.WriteLine("Цена: {0}", pizza2.GetCost());

Pizza pizza3 = new BulgerianPizza();
pizza3 = new TomatoPizza(pizza3);
pizza3 = new CheesePizza(pizza3);//болгарская пиццы с томатами и сыром
Console.WriteLine("Название: {0}", pizza3.Name);
Console.WriteLine("Цена: {0}", pizza3.GetCost());

Console.ReadLine();
```

В качестве компонента здесь выступает абстрактный класс Pizza, который определяет базовую функциональность в виде свойства Name и метода GetCost(). Эта функциональность реализуется двумя подклассами ItalianPizza и BulgerianPizza, в которых жестко закодированы название пиццы и ее цена.

Декоратором является абстрактный класс PizzaDecorator, который унаследован от класса Pizza и содержит ссылку на декорируемый объект Pizza. В отличие от формальной схемы здесь установка декорируемого объекта происходит не в методе SetComponent, а в конструкторе.

Отдельные функциональности - добавление томатов и сыры к пиццам реализованы через классы TomatoPizza и CheesePizza, которые обертывают объект Pizza и добавляют к его имени название добавки, а к цене - стоимость добавки, то есть переопределяя метод GetCost и изменяя значение свойства Name.

Благодаря этому при создании пиццы с добавками произойдет ее обертывание декоратором:

```
Pizza pizza3 = new BulgerianPizza();
pizza3 = new TomatoPizza(pizza3);
pizza3 = new CheesePizza(pizza3);
```

Сначала объект BulgerianPizza обертывается декоратором TomatoPizza, а затем CheesePizza. И таких обертываний мы можем сделать множество. Просто достаточно унаследовать от декоратора класс, который будет определять дополнительный функционал.

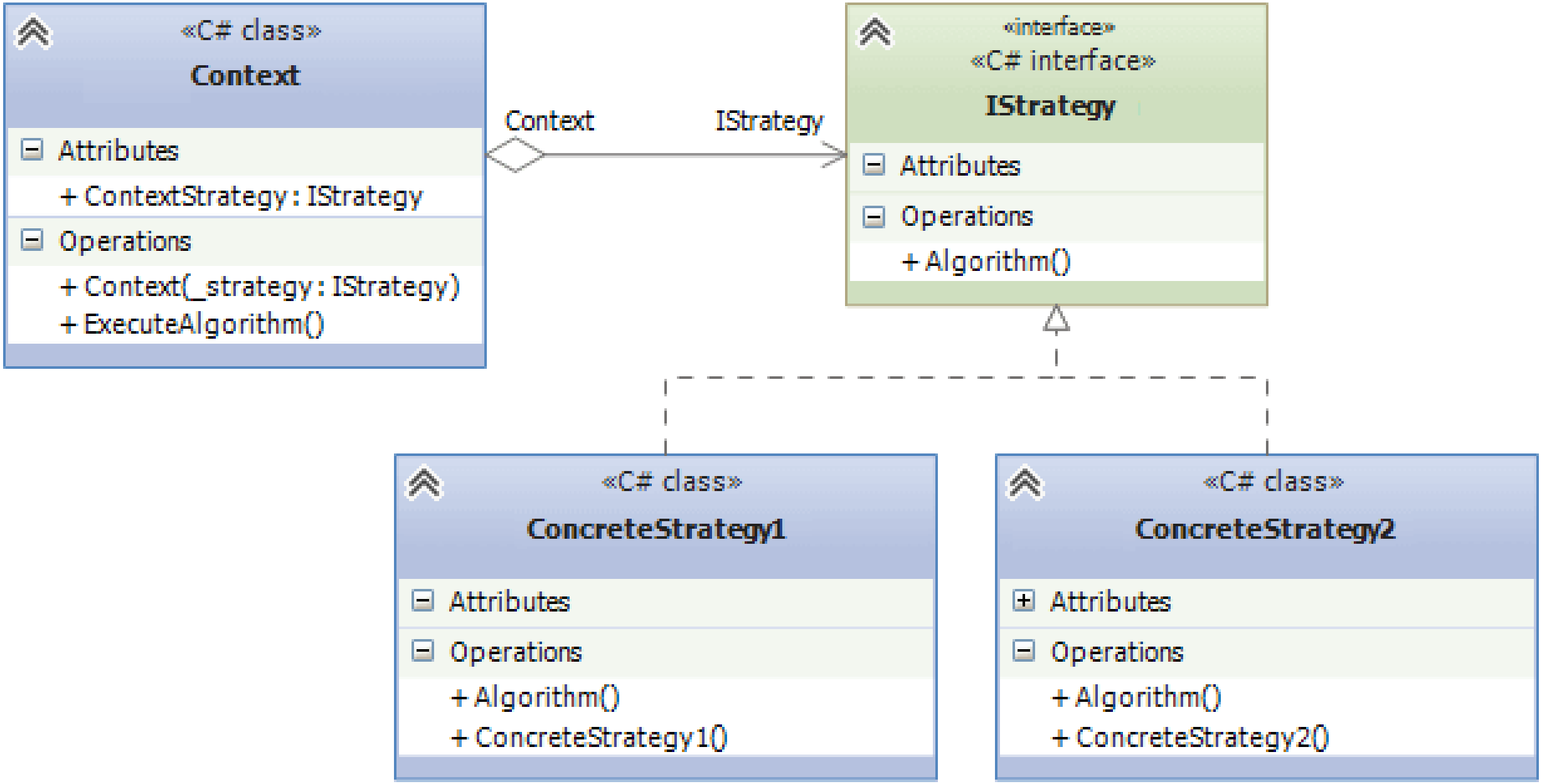
А если бы мы использовали наследование, то в данном случае только для двух видов пицц с двумя добавками нам бы пришлось создать восемь различных классов, которые бы описывали все возможные комбинации. Поэтому декораторы являются более предпочтительным в данном случае методом.

Паттерн Стратегия

Паттерн Стратегия (Strategy) представляет шаблон проектирования, который определяет набор алгоритмов, инкапсулирует каждый из них и обеспечивает их взаимозаменяемость. В зависимости от ситуации мы можем легко заменить один используемый алгоритм другим. При этом замена алгоритма происходит независимо от объекта, который использует данный алгоритм.

Когда использовать стратегию?

- Когда есть несколько родственных классов, которые отличаются поведением. Можно задать один основной класс, а разные варианты поведения вынести в отдельные классы и при необходимости их применять.
- Когда необходимо обеспечить выбор из нескольких вариантов алгоритмов, которые можно легко менять в зависимости от условий.
- Когда необходимо менять поведение объектов на стадии выполнения программы.
- Когда класс, применяющий определенную функциональность, ничего не должен знать о ее реализации.
- *Когда много IF ELSE SWITCH*


«C# class»
Context
Attributes
+ ContextStrategy : IStrategy
Operations
+ Context(_strategy : IStrategy)
+ ExecuteAlgorithm()
Context
IStrategy
«interface»
«C# interface»
IStrategy
Attributes
Operations
+ Algorithm()
«C# class»
ConcreteStrategy1
Attributes
Operations
+ Algorithm()
+ ConcreteStrategy1()
«C# class»
ConcreteStrategy2
Attributes
Operations
+ Algorithm()
+ ConcreteStrategy2()

```
public interface IStrategy
{
    void Algorithm();
}

public class ConcreteStrategy1 : IStrategy
{
    public void Algorithm()
    {}
}

public class ConcreteStrategy2 : IStrategy
{
    public void Algorithm()
    {}
}
```

```
public class Context
{
    public IStrategy ContextStrategy { get; set; }

    public Context(IStrategy _strategy)
    {
        ContextStrategy = _strategy;
    }

    public void ExecuteAlgorithm()
    {
        ContextStrategy.Algorithm();
    }
}
```

Рассмотрим **пример**, в котором есть несколько алгоритмов сортировки, и мы хотим использовать паттерн "Стратегия" для их реализации.

1. Создаём общий интерфейс для всех сортировок:

```
public interface ISortStrategy
{
    void Sort(int[] array);
}
```

Создаём конкретную сортировку («Пузырьком»):

```
public class BubbleSort : ISortStrategy{
    public void Sort(int[] array)
    {
        for (int i = 0; i < array.Length - 1; i++)
        {
            for (int j = 0; j < array.Length - i - 1; j++)
            {
                if (array[j] > array[j + 1])
                {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                }
            }
        }
    }
}
```

Создаём 2-ю конкретную сортировку («Быстрая сортировка»):

```
public class QuickSort : ISortStrategy
{
    public void Sort(int[] array)
    {
        QuickSortAlgorithm(array, 0, array.Length - 1);
    }

    private void QuickSortAlgorithm(int[] array,
                                    int low, int high)
    {
        if (low < high)
        {
            int pi = Partition(array, low, high);
            QuickSortAlgorithm(array, low, pi - 1);
            QuickSortAlgorithm(array, pi + 1, high);
        }
    }
```

```
private int Partition(int[] array,
                                int minIndex, int maxIndex)
{
    var pivot = minIndex - 1;
    for (var i = minIndex; i < maxIndex; i++)
    {
        if (array[i] < array[maxIndex])
        {
            pivot++;
            Swap(ref array[pivot], ref array[i]);
        }
    }

    pivot++;
    Swap(ref array[pivot], ref array[maxIndex]);
    return pivot;
}
```

```
private void Swap(ref int x, ref int y)
{
    var t = x;
    x = y;
    y = t;
}
```

Шаг 3: Реализация контекста

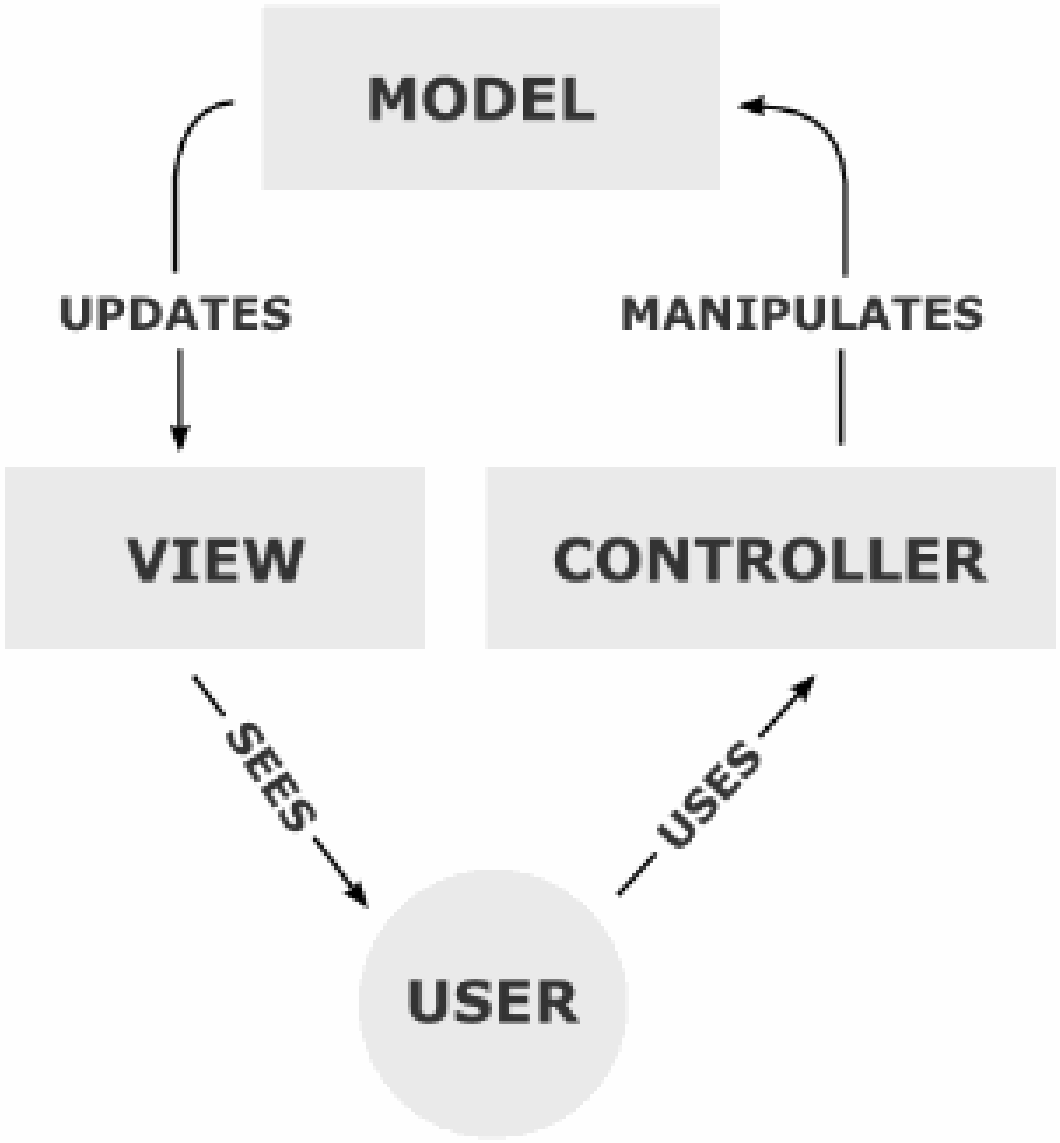
```
public class SortContext {
    private ISortStrategy _sortStrategy;

    public SortContext(ISortStrategy sortStrategy)
    {
        _sortStrategy = sortStrategy;
    }

    public void SetSortStrategy(ISortStrategy sortStrategy)
    {
        _sortStrategy = sortStrategy;
    }

    public void Sort(int[] array)
    {
        _sortStrategy.Sort(array);
    }
}
```

Шаг 4: Использование паттерна "Стратегия"

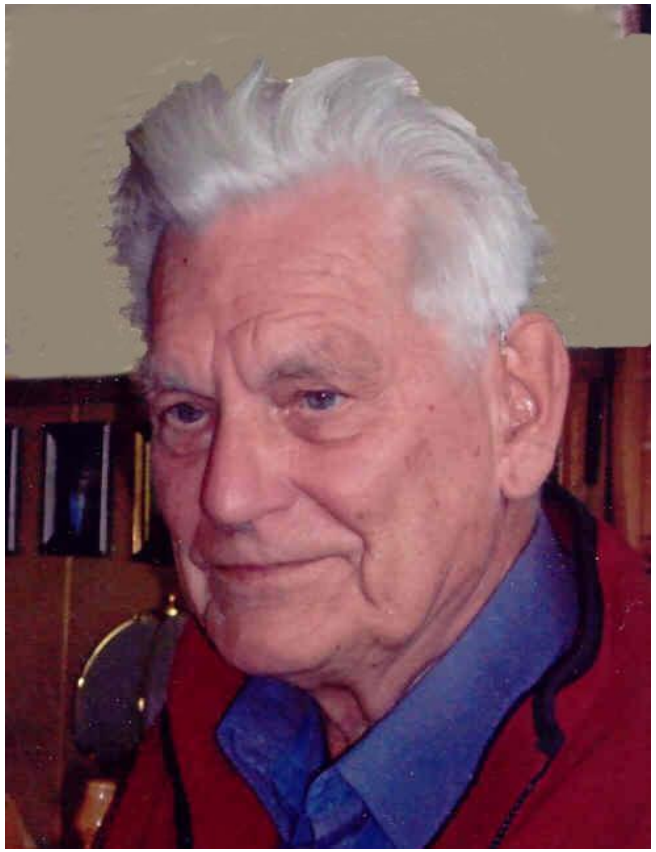
```
class Program{
    static void Main(string[] args){
        int[] array = { 5, 3, 8, 6, 2 };

        SortContext context = new SortContext(
                                        new BubbleSort());
        Console.WriteLine("Сортировка пузырьком:");
        context.Sort(array);
        Console.WriteLine(string.Join(", ", array));
         // Изменение стратегии на быструю сортировку
        array = new int[] { 5, 3, 8, 6, 2 };
        context.SetSortStrategy(new QuickSort());
        Console.WriteLine("Быстрая сортировка:");
        context.Sort(array);
        Console.WriteLine(string.Join(", ", array));
    }
}
```

Паттерн MVC

Model, View, Controller (MVC) — это шаблон (паттерн) программирования, разделяющий архитектуру приложения на три модуля: модель (Model), представление (View), контроллер (Controller). Простыми словами, он позволяет изменять каждый компонент независимо друг от друга для простой разработки и поддержки приложений (очень часто веб).



Концепцию MVC предложил в конце 1970-х годов сотрудник Xerox Трюгве Реекскауг. Она была реализована в языке программирования Smalltalk-80. Окончательную версию шаблона опубликовали только 10 лет спустя в журнале Technology Object. Концепция стала популярна с появлением быстро развертываемых фреймворков и интерактивных веб-приложений.



Модель (Model)

Это основная логика приложения. Отвечает за данные, методы работы с ними и структуру программы. Модель реагирует на команды из контроллера и выдает информацию и/или изменяет свое состояние. Она передает данные в представление.

#SkillFactory

Модель (Model)

Модель в приложении MVC представляет состояние приложения и бизнес-логику или операций, которые должны в нем выполняться. Бизнес-логика должна быть включена в состав модели вместе с логикой реализации для сохранения состояния приложения. Как правило, строго типизированные представления используют типы ViewModel, предназначенные для хранения данных, отображаемых в этом представлении. Контроллер создает и заполняет эти экземпляры ViewModel из модели.

#MSDN Microsoft

Представление (View)

Задача компонента — визуализация информации, которую он получает от модели. View отображает данные на уровне пользовательского интерфейса. Например, в виде таблицы или списка. Представление определяет внешний вид приложения и способы взаимодействия с ним.

#SkillFactory

Представление (View)

Представления отвечают за представление содержимого через пользовательский интерфейс. Представления должны иметь минимальную логику, которая должна быть связана с представлением содержимого. Если есть необходимость выполнять большую часть логики в представлении для отображения данных из сложной модели, рекомендуется воспользоваться компонентом представления, ViewModel или шаблоном представления, позволяющими упростить представление.

#MSDN Microsoft

Контроллер (Controller)

Он обеспечивает взаимодействие с системой: обрабатывает действия пользователя, проверяет полученную информацию и передает ее модели. Контроллер определяет, как приложение будет реагировать на действия пользователя. Также контроллер может отвечать за фильтрацию данных и авторизацию.

#SkillFactory

Контроллер (Controller)

Контроллеры — это компоненты для управления взаимодействием с пользователем, работы с моделью и выбора представления для отображения. В приложении MVC представление служит только для отображения информации. Обработку введенных данных, формирование ответа и взаимодействие с пользователем обеспечивает контроллер. В структуре MVC контроллер является начальной отправной точкой и отвечает за выбор рабочих типов моделей и отображаемых представлений (именно этим объясняется его название — он контролирует, каким образом приложение отвечает на конкретный запрос).

#MSDN Microsoft

Компоненты модели различаются степенью зависимости друг от друга и ограничениями:

1. модель не зависит от представления и контроллера;
2. представление может обращаться к модели за данными и событиями, но не может ее менять;
3. контроллер не может отображать данные, но способен менять модель в зависимости от действий пользователя.

Как работает MVC

Разберем на реальном примере. Условная физическая модель MVC-архитектуры — персональный компьютер, в котором:

модель — системный блок, в котором происходит обработка команд и хранятся системные и пользовательские файлы;

представление — монитор, на котором визуализируется работа системного блока;

контроллер — клавиатура или мышь. С их помощью пользователь вводит команды.

На этом примере легче понять зависимость компонентов друг от друга:

если заменить системный блок (без переноса старых данных), но оставить старый монитор и клавиатуру, пользователь получит другой компьютер с новой ОС, файлами, системными характеристиками, драйверами и т.д.;

если поменять монитор и клавиатуру, но оставить старый системный блок, человек будет использовать прежнюю ОС, файлы, системные характеристики. Изменения коснутся только некоторых драйверов.

Пример проекта Windows Forms с использованием MVC

Описание проекта

В этом примере мы создадим простое приложение для управления списком контактов. Приложение будет позволять пользователю добавлять, удалять и отображать контакты.

Структура проекта

1. **Model**: класс **Contact**, который будет представлять контакт.
2. **View**: форма **MainForm**, которая будет отображать интерфейс пользователя.
3. **Controller**: класс **ContactController**, который будет управлять взаимодействием между моделью и пред-ставлением.

Шаг 1: Создание модели

Создадим класс Contact в папке Model проекта.

```
// Model/Contact.cs
public class Contact
{
    public string Name { get; set; }
    public string PhoneNumber { get; set; }

    public Contact(string name, string phoneNumber)
    {
        Name = name;
        PhoneNumber = phoneNumber;
    }
}
```

Шаг 2: Определение интерфейсов

Создадим интерфейсы для представления и контроллера.

```
// Interfaces/IContactView.cs
using System.Collections.Generic;

public interface IContactView
{
    event Action<string, string> ContactAdded;
    event Action<string> ContactRemoved;

    void UpdateContactList(List<Contact> contacts);
}
// Interfaces/IContactController.cs
public interface IContactController
{
    void AddContact(string name, string phoneNumber);
    void RemoveContact(string name);
}
```

Шаг 3: Создание представления

Создадим форму MainForm, которая будет содержать элементы управления для отображения и управления контактами.

```
// View/MainForm.cs
public partial class MainForm : Form, IContactView
{
                    // Имя и номер телефона
    public event Action<string, string> ContactAdded;
                    // Имя
    public event Action<string> ContactRemoved;

    public MainForm()
    {
        InitializeComponent();
    }
```

```
public void UpdateContactList(List<Contact> contacts)
{
    listBoxContacts.Items.Clear();
    foreach (var contact in contacts)
    {
        listBoxContacts.Items.Add(
                $"{contact.Name} - {contact.PhoneNumber}");
    }
}

private void buttonAdd_Click(object sender, EventArgs e)
{
    ContactAdded?.Invoke(textBoxName.Text,
                         textBoxPhoneNumber.Text);
}
```

```
    private void buttonRemove_Click(object sender, EventArgs e)
    {
        if (listBoxContacts.SelectedItem != null)
        {
            string selectedContact =
                        listBoxContacts.SelectedItem.ToString();

            string name = selectedContact.Split('-')[0].Trim();
            ContactRemoved?.Invoke(name);
        }
    }
}
```

Шаг 4: Создание контроллера

Создадим класс контроллера, который будет управлять логикой приложения.

```
// Controller/ContactController.cs
public class ContactController : IContactController
{
    private List<Contact> contacts;
    private IContactView view;

    public ContactController(IContactView view)
    {
        this.view = view;
        contacts = new List<Contact>();

        this.view.ContactAdded += OnContactAdded;
        this.view.ContactRemoved += OnContactRemoved;
    }
```

```
private void OnContactAdded(string name,
                            string phoneNumber) {
    AddContact(name, phoneNumber);
}

private void OnContactRemoved(string name) {
    RemoveContact(name);
}

public void AddContact(string name, string phoneNumber) {
    contacts.Add(new Contact(name, phoneNumber));
    view.UpdateContactList(contacts);
}

public void RemoveContact(string name) {
    contacts.RemoveAll(c => c.Name == name);
    view.UpdateContactList(contacts);
}
```

Шаг 5: Связывание компонентов в Main

Теперь свяжем все компоненты в методе Main.

```
// Program.cs
static class Program
{
    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);

        IContactView view = new MainForm();
        IContactController controller =
                                    new ContactController(view);

        Application.Run((Form)view);
    }
}
```

Ссылки на литературу

Паттерны: https://metanit.com/sharp/patterns/1.1.php